

Transformer Architecture: Encoder

Contents

- ✓ Motivation for Transformers
- ✓ Introduction to Transformers
- ✓ Function of Encoder and Decoder in a Transformer
- ✓ Positional encoding
- ✓ Numerical in positional encoding
- ✓ Encoder Architecture
- ✓ Residual Connection and Layer normalization
- ✓ Position-wise Feed-Forward Network

Motivation for Transformers

Limitations of Sequence-2-Sequence models like RNNs, LSTMs and GRUs

- ✓ Sequential computation makes training slow
- ✓ Difficulty in parallelization
- ✓ Long-range dependency problem

Although attention improved Seq2Seq models, they still relied on recurrence, making training sequential.

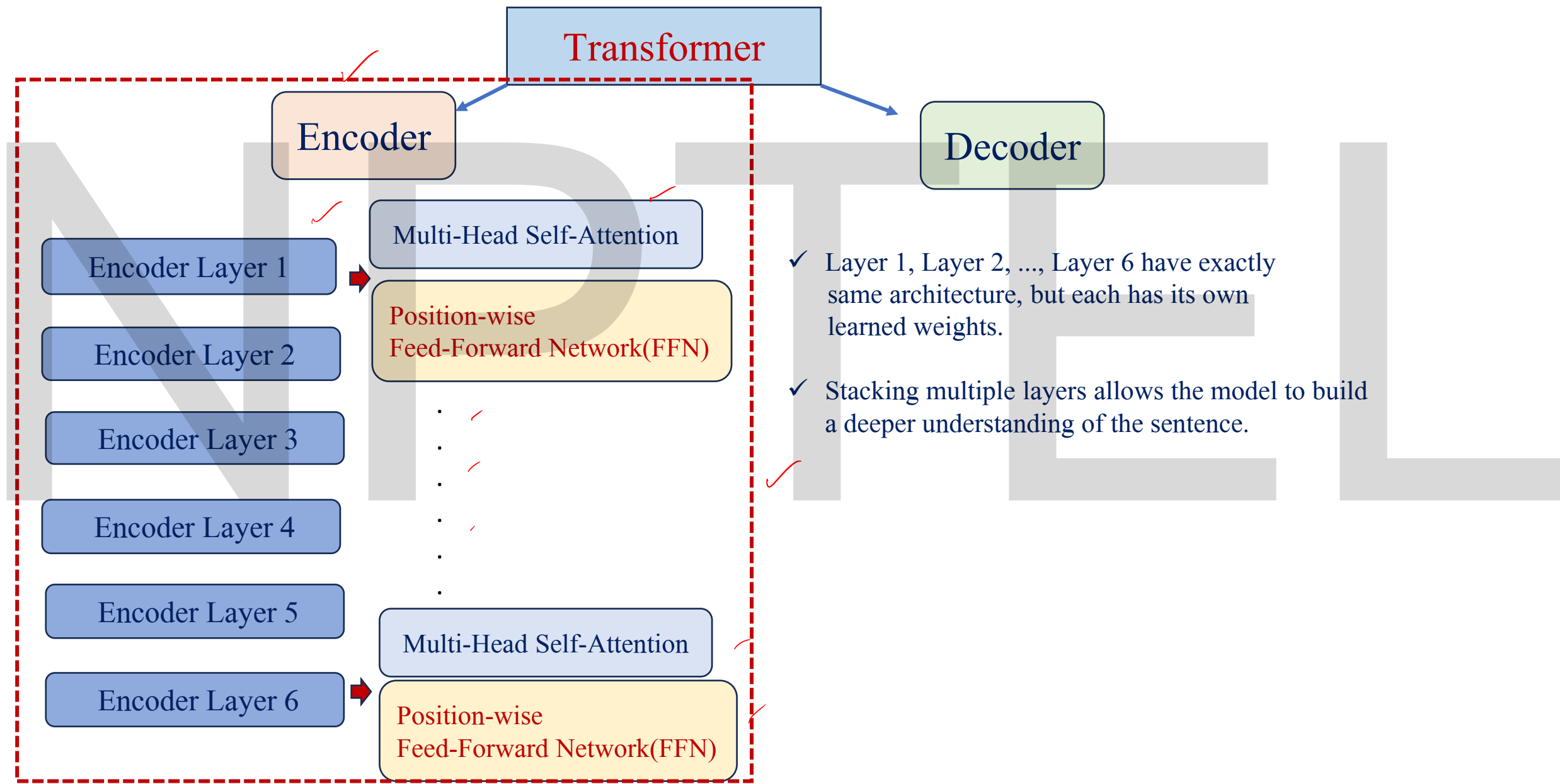
Can we remove recurrence entirely?

The Transformer is based only on attention, eliminating recurrence and convolutions.

Ref:

Vaswani et al., Attention Is All You Need, Advances in Neural Information Processing Systems (NIPS 2017)

Introduction to Transformers



Function of Encoder and Decoder in a Transformer

Ex: Language translation

Encoder → Understand the input.
Decoder → Generate the output.

ENCODER

- Processes all input tokens simultaneously
- Generates **contextual** representation for every token

DECODER

- Does **not** generate the entire sentence at once.
- Generates one word at a time during inference.

At every step, the decoder

- ✓ looks at the words it has already generated
- ✓ looks at the encoder output
- ✓ then predicts the next word.

Positional Encoding

In RNNs/LSTMs, words are processed one after another.

The → cat → sat → on → the → mat

The hidden state carries information from previous words, so the model naturally knows the order of the words.

But Transformers process all words simultaneously

- All words are converted into embeddings and fed to the model at the same time.
- **Positional encoding** injects information about the order of tokens into their embeddings.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Even dimensions use sine and odd dimensions use cosine.

Positional Encoding- Numerical Example

Assume embedding dimension $d_{\text{model}} = 4$.

$[-, -, -, -]$

original paper $d_{\text{model}} = 512$.

The cat sat on the mat

pos:-

0 1 2 3 4 5

pos=2

[finding the PE for the token/word "sat"]

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right);$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$\text{For } i=0 \quad PE(2, 0) = \sin\left(\frac{2}{10000^0}\right) = \sin(2) = 0.9093.$$

$$PE(2, 1) = \cos\left(\frac{2}{10000^0}\right) = \cos(2) = -0.4161$$

Positional Encoding- Numerical Example

$$i=1 \quad PE(2,2) = \sin\left(\frac{2}{10000^{2/42}}\right) = \sin\left(\frac{2}{100}\right) = \sin(0.02) \approx 0.0200$$

$$PE(2,3) = \cos\left(\frac{2}{10000^{2/42}}\right) = \cos\left(\frac{2}{100}\right) = \cos(0.02) \approx 0.9998$$

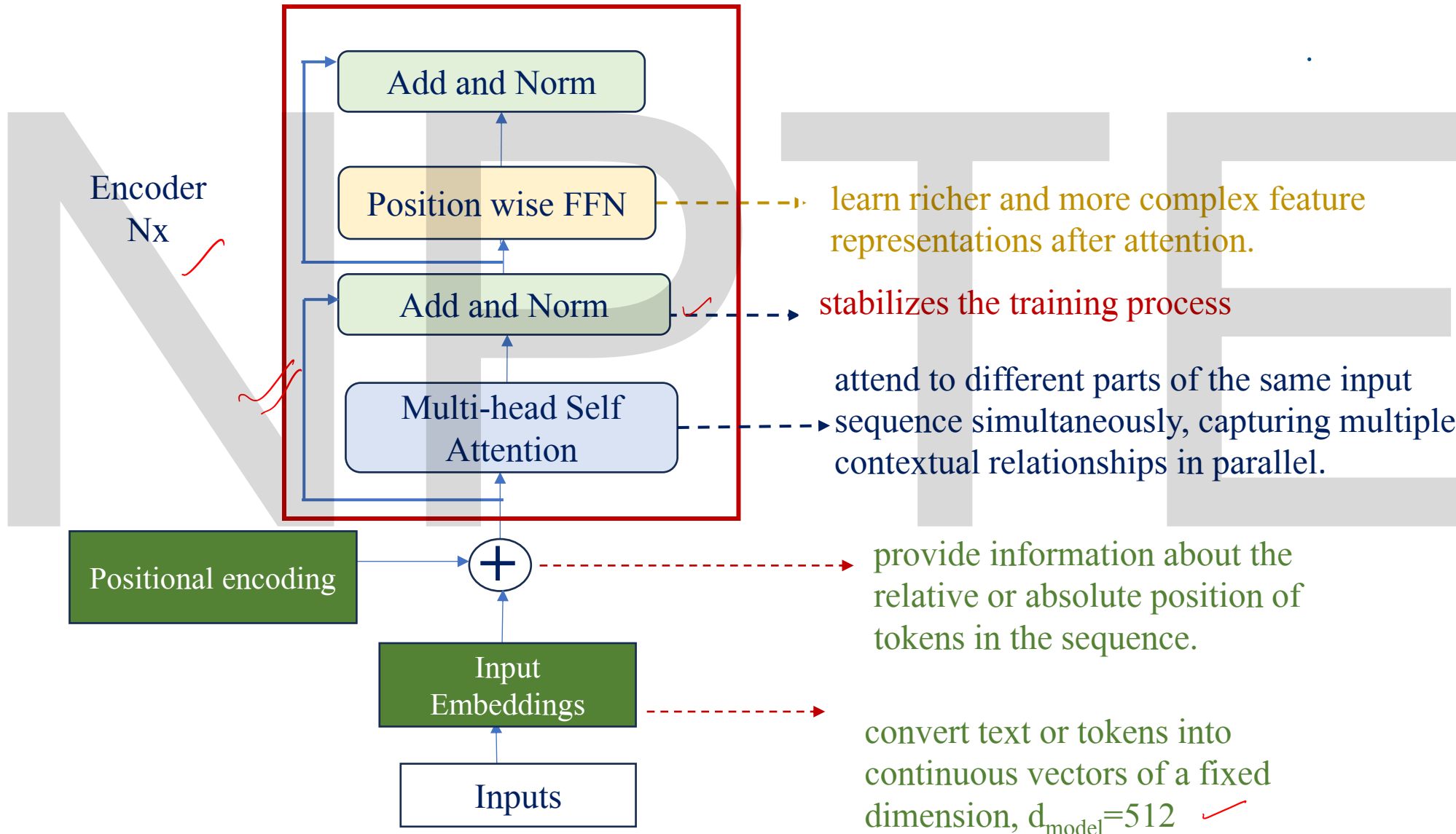
$$PE(2) = [0.9093, -0.4161, 0.0200, 0.9998]$$

gs vector

embedding for sat
Final embedding

$$\begin{bmatrix} x_1 & x_2 & x_3 & x_4 \\ 0.9093 + x_1 & -0.4161 + x_2 & 0.0200 + x_3 & 0.9998 + x_4 \end{bmatrix}$$

Encoder Architecture



Residual Connection and Layer Normalization

Why Add the Original Input?

- ✓ The sub layer may completely change the representation.
- ✓ Important information may disappear.

The Transformer preserves the original representation using residual connections

Why Layer Norm?

Different neurons can produce outputs with different scales.

Layer Norm,

- ✓ Stabilizes activations and improves optimization
- ✓ Makes training faster

word embeddings with positional encoding

x

Sublayer

Layer Normalization

Layer Norm ($x + \text{Sublayer}(x)$)

Multi head self attention

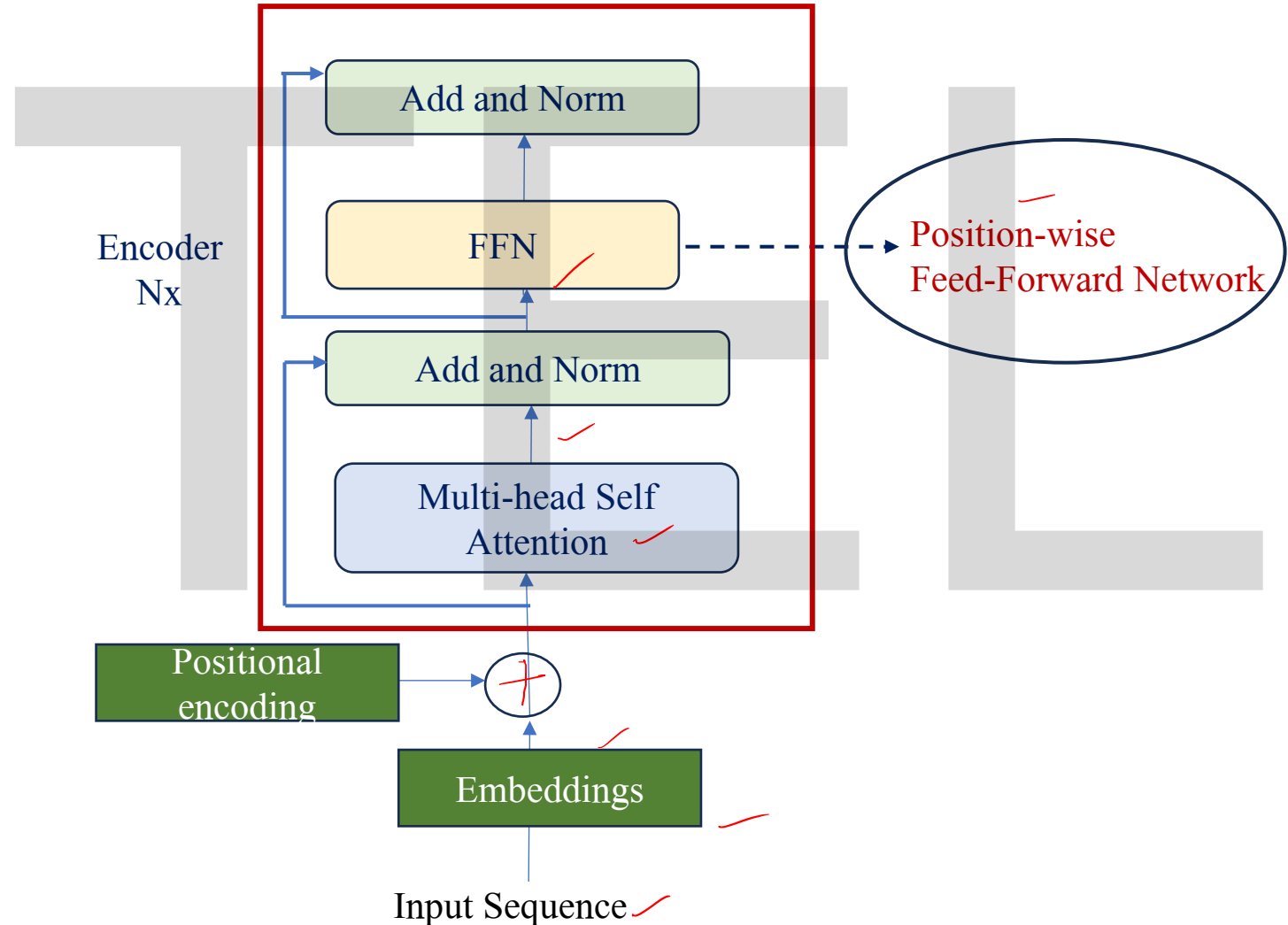
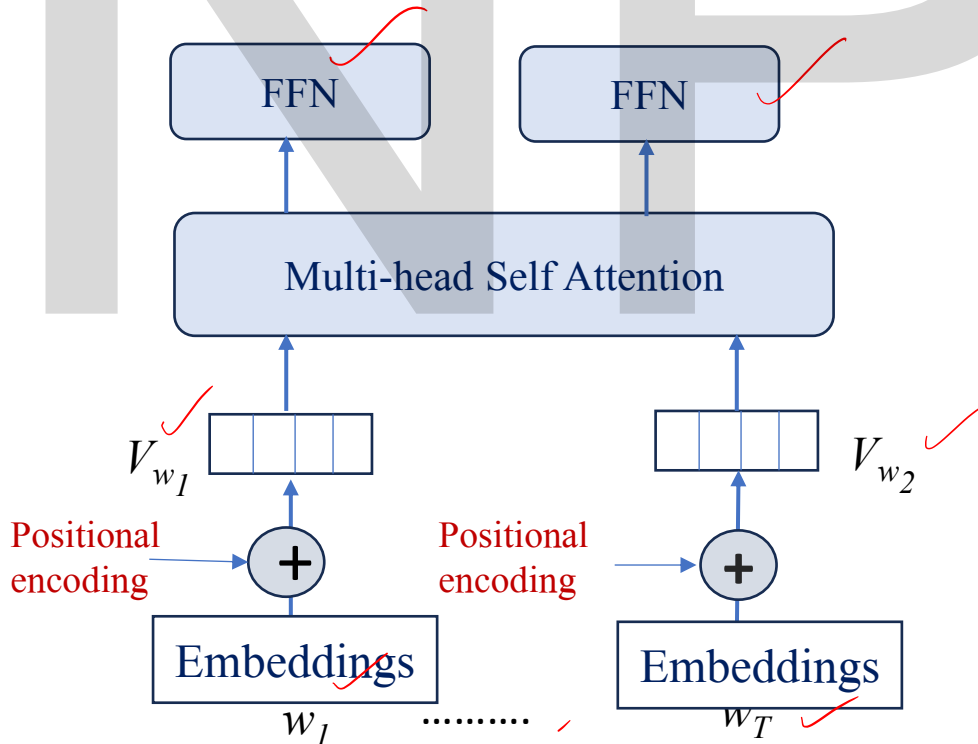
Position-wise Feed-Forward Network (FFN)

Attention layer gathers information from all the words in the sentence.



✓ Each token is transformed independently using the same feed-forward network.

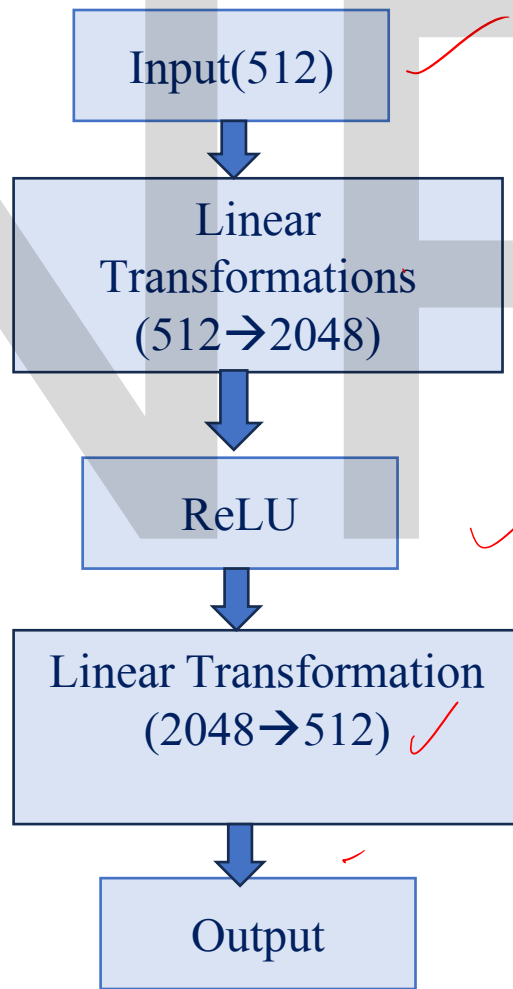
✓ This is done by position-wise FFN



Architecture of FFN

Consists of

- ✓ Two linear transformation
- ✓ ReLU activation between the two linear transformations



$$\text{FFN}(x) = \max(0, \underbrace{xW_1 + b_1}) \underbrace{W_2 + b_2}$$

input
first linear transformation.
 $W_2, b_2 \Rightarrow$ 2nd linear transformation.

NPTEL

Summary

NLP TEL

Next Session

Transformer Architecture- Decoder